

2833225(Mashudu Lishivha Retrospective): Sprint 1

For this project we all started by discussing what tools we needed and how we are supposed to go about building it. It was kind of hard to come up with ideas. We wrote users stories and they were uploaded on Taiga and assigned to us by Boitumelo. I was assigned to work on user story two: “As an Admin, I want to create a stokvel group so that I can set up my savings group. “And user story three: “As an Admin, I want to invite members so that they can join my stokvel group.” Along with Boitumelo.

I created the database using Postgresql Supabase platform and used the Google cloud for the google auth so that users will be able to sign in with emails. This involved designing the initial SQL schema for the users and Stokvel Groups tables and ensuring the connection strings were properly secured. The database task is TG-5 on taiga. The database for this sprint was my responsibility, and it was confusing and hard to use it, as the platform was new to me. It was hard because the person who built the sign in page, we struggled with letting users sign in with their emails but succeeded at the end. And, I had to work on the file for the database which is subabase-client.js. I built the “Create-group” frontend for the admin only which is TG-6, where we supposed to add members by inviting them by sending links. I struggled with the backend of it and the button doesn’t create groups, we can only see its display. So, the backend of the invite form is also not complete, which is TG-10.

We all tried to assist each other where we can. They are something things which don’t work, we plan on improving ourselves and learning. For the TG-10 (configure email to include group link), while the UI was ready, I hit a major roadblock with the link-sending logic. I struggled with the integration between our frontend and the Supabase Auth service; the “Create group” button didn’t create groups, and the invitation links were not being generated. This was the hardest thing for me, but I managed to let the user’s details be saved in the database and their roles also. What we could have fixed is assign the user stories that we would be able to complete.

2833225(Mashudu Lishivha Retrospective 2)

For sprint 2 we had a lot of work to complete since we did not finish sprint 1 tasks. All of us as a group had to first have a meeting discussing what was incomplete and what we had to do to complete the work. We gave ourselves a timeframe to complete sprint 1 so we can continue with sprint 2. I managed to complete the “create group” task which is TG-6 in Taiga, and it was the hardest thing to fix, and without it, the sprint 2 task would not be completed. I fixed it by using modifying the “createGroup” function, “getMyGroups” in supabase-client file, and also modifying the groups.js file. It can create a new group, and the group is displayed on the admin dashboard and the dropdown on the “Invite members” page.

So, for sprint two I had to make the logout button work which is one of the user stories on Taiga (#31) and task TG-32. It was easier, since I fixed it when fixing the “create group” functions and files and on GitHub it’s fixed on TG-6 comment. I had to edit the “main.js”, “auth.js” and “supabase-client.js” files. I also worked on task TG-26. It is for updating the accept invitation function on the database and I didn’t struggle that much with it, since I had the Row Level Security (RLS) policies guiding me in the database. Implementing these RLS policies was a key architectural decision our team made to ensure that group data remains private and secure, which is vital for a financial platform like ours. I also worked on the last user story, that allows members to view the group that they were invited to. It was task TG-36 where I added a new html file for “My-groups.js”, updated “main.js”, “dashboards.js”, auth.js”. Since I fixed the “create groups” functions, I did not struggle that much with it for displaying the groups, but I had to read the files, debug the page and edit the supabase-client file so that the groups could appear.

We had a short period of time for sprint 2 and that is why we decided that we will use user stories that are connected to the sprint 1 task and just enhance them and also add few new things. The user stories were simple, as we need to complete previous work. For the next sprint we will start working on the payments related user stories and we plan to improve the website and its visuals so that it’s visually appealing and easy to navigate. What I learnt is that we should try coming up with ideas, tasks and user stories that we will be able to complete at our pace, also trying to complete the project.

2833225 Sprint 3 Retrospective

For sprint 3, we had so much to do since we needed to drop the mock data that was being used, and switch to real data, that would be stored in the database. We completed all tasks for this sprint. My biggest challenge for this sprint was the database RLS issues and some of the functions on the supabase-client.js. My task on taiga is TG-59, where I have to update the file with newly added functionalities by creating the functions which would connect to each new JavaScript file and also update the database in supabase. And also, TG-60, where I share the task with Boitumelo by updating routing and him, responsible for coverage tests. The challenge I faced was that none of the Sprint 3 features were loading member data correctly. I traced the issue to the group_members table having a conflicting foreign key constraint and missing RLS policies on the profiles table. I fixed it by dropping the conflicting constraint and adding the correct RLS policy so authenticated users could view all profiles.

When teammates added meetings.js, payouts.js, analytics.js, the whole app crashed because main.js didn't import them, causing the sign in button to stop working. I then added the imports and route mappings for all three new pages on the main.js file.

The member data wasn't loading because of missing RLS policies. I then added the correct policy so authenticated users can view profiles. After login, all users were going to the same dashboard regardless of role. I updated checkAuth() to read role from localStorage and redirect correctly on auth.js.

For this sprint I learned that Supabase's Row Level Security blocks everything by default and you have to explicitly write policies for every table and operation. A missing policy returns empty data silently with no error message which made debugging very frustrating and time consuming. Working on main.js taught me how fragile ES module imports can be. When payouts.js was missing the entire application stopped working including the login. And working on the supabase-client.js, I had to communicate clearly about function names, parameters and return values so they could build their features without being blocked by me.

If I were to do Sprint 3 again, I would set up the Supabase schema and RLS policies at the very beginning of the sprint before any feature development started. We lost several days of progress because teammates were building features that couldn't load data due to missing policies. Setting up the database layer first would have unblocked everyone much earlier. I would add better error handling to the database functions from the start.

2833225 Sprint 4 retrospective

On sprint 4, we had 6 user stories to complete so that our app would be complete. I did a user story on task TG-71 and TG-66 on Taiga, which involves creating a profile page with profile.html and profile.js. I also had to update the database and a line on supabase file, so that new users who log in should be assigned the admin role instead of member so that it is easier to test. I also did the routing so that new pages appear.

The biggest challenge was the database trigger. When I first tried to create the trigger to assign admin role, it was failing because the profiles table did not have a role column. The role is stored in a separate user_roles table. I eventually traced it to the trigger trying to insert a column that didn't exist, then fixed it by splitting the logic, inserting into profiles for personal info and inserting into user_roles for the role.

Building the profile page was also a little bit complex. The main challenge was getting the user stats to calculate correctly. The compliance rate was shown as 0% for all users even when they had completed contributions. I traced the issue to the fact that `getMyContributions()` was returning all contributions including ones from groups the user had left, so the total count was inflated, and the completed count was low making the percentage incorrect. I fixed it by filtering contributions to only count those belonging to the user's current active groups before calculating the compliance rate.

Another problem was the sidebar not updating after the user saved their name. Even though the name was updating in Supabase and in localStorage the sidebar was still showing the old name because `checkAuth()` reads from localStorage at page load and doesn't re-run after a save. I fixed this by manually updating the sidebar DOM elements directly after a successful save without relying on `checkAuth()` to re-run.

I learned how Supabase database triggers work in practice, specifically that triggers fire on `auth.users` inserts and run as `SECURITY DEFINER` which means they bypass RLS. I also learned that having two triggers on the same table event can cause conflicts and it is better to combine related logic into one function. What I would do differently would be to set up and test the database triggers at the very beginning of the project. The trigger issues blocked new user when testing for the entire Sprint 4 testing phase. I would also write the trigger logic in a separate migration file that is version controlled so the team can see exactly what changed in the database without having to read through the SQL editor history.